

COM3110 Text Processing Document Retrieval Assignment

Vivek Vijay Choradia

University of Sheffield

16 November 2023

This assignment includes the implementation of *Retrieve.py*, a basic information retrieval system (IR) designed to extract relevant information from a collection of documents recording publications in the CACM¹. The script employs three weighting systems - binary, term frequency (tf), and term frequency - inverse document frequency (tfidf) - to retrieve information. The report provides insights into the implementation and performance scores of each system.

1 Implementation of Weighting System

Each weighting system implemented aims to capture different aspects of term importance in the document collection, providing the "benchmark" or "standard test set" for evaluating IR systems.

1.1 Binary:

The binary system is implemented with the term weight being exclusively either 1 or 0. For every document, $q_i d_i$ is the number of common terms that appear in both the query and the document and d_i^2 is the number of terms in the document. The similarity score is calculated as the number of common terms divided by the square root of the number of terms in the relevant document. The similarity score of a document is higher if it contains more terms from the query.

1.2 Term Frequency (TF):

The TF system is implemented considering the frequency of terms in both the query and the document. To reduce the retrieval time, the size of each document vector $\sqrt{\sum_{i=1}^n d_i^2}$ is calculated before the loop. The similarity score is computed by summing the product of term frequencies in the query and the document divided by the Euclidean norm of the document vector. The similarity score of a document is higher if it contains more terms with higher frequency in the document and the query.

1.3 Term Frequency - Inverse Document Frequency (TFIDF):

TF-IDF is a widely used weighting scheme that considers the frequency of a term in a document (TF) and its importance in the entire document collection (IDF). To prevent repeating the same calculation in the loop, the TFIDF value of each term in the query is calculated beforehand. For every document, the $q_i d_i$ value is the multiplication of the TFIDF of the query term with the TFIDF of the query term in the document. The similarity score is calculated by summing the product of TF-IDF values for common terms in the query and the document divided by the Euclidean norm of the document vector.

An attempt to alternative term weight was made using maximum tf normalisation. The term weight was calculated using the formula 1.

To rank the documents, a similarity score was computed for each query. The similarity score was computed as the cosine angle between the vectors of term weights (equation mentioned in Formula 2).

2 Result

To further the performances, a combination of preprocessing was used - stemming and stoplisting. In layman's terms, stemming is nothing but reducing words to their base or root form and stoplisting is filtering out common words which are insignificant for querying.

¹CACM: Communications of the Association for Computing Machinery

To compare the results of using different implementation systems and pre-processing, the F-measure² value was used. Table 1 and Figure 1 shows a summary of results. Further information on results is available in the Appendix Table 2 and Figure 2, which also provides insights on time retrieval.

	Binary	TF	TFIDF
None	0.07	0.06	0.17
Stemming only	0.08	0.09	0.21
Stoplisting only	0.12	0.14	0.17
Both	0.14	0.17	0.23

Table 1: F-measures of IR system under a range of configurations.

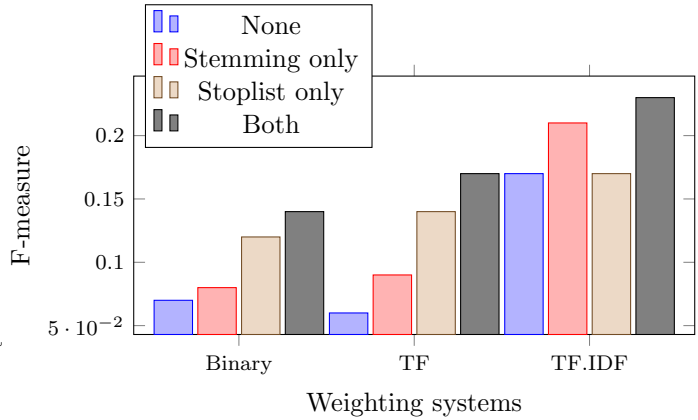


Figure 1: F-measures of IR system under a range of configurations.

3 Discussion and Conclusion

Based on the results shown in Table 1 and Figure 1, here are a few evaluations:

- **TFIDF outperforms binary and TF:** The results show that the TFIDF weighting system has the highest F-measure among the three systems, indicating that it retrieves the most relevant and complete information for the queries. This suggests that the TFIDF system captures the importance of terms in both the document and the collection better than the binary and TF systems, which only consider the presence or frequency of terms in the document.
- **Stemming and stoplisting improve performance:** The results also show that the performance of all three systems improves when stemming and stoplisting are applied as preprocessing steps. Stemming reduces the number of distinct terms in the collection by converting words to their root forms, while stoplisting removes common and insignificant words that may not contribute to the query. These steps help to reduce the noise and increase the precision of the IR system.
- **Trade-off between relevance and speed:** The results reveal a trade-off between the relevance and the speed of the IR system. The TFIDF system, which has the highest F-measure, also has the longest retrieval time among the three systems. This is because the TFIDF system requires more calculations to compute the term weights and the similarity scores. On the other hand, the binary system, which has the lowest F-measure, also has the shortest retrieval time. This is because the binary system only needs to check whether a term is present or not in the document, which is a simple operation.
- **Optimizing the IR system:** The findings of this assignment suggest that there is no single optimal weighting system for IR, but rather a balance between the effectiveness and the efficiency of the system. Depending on the application and the user's needs, different weighting systems and preprocessing steps may be more suitable. For example, if the user values speed over relevance, the binary system with stoplisting may be preferred. However, if the user values relevance over speed, the TFIDF system with both stemming and stoplisting may be preferred.

In conclusion, this assignment demonstrates the implementation and evaluation of three weighting systems for information retrieval: binary, TF, and TFIDF. The results show that TFIDF performs the best in terms of relevance, while binary is the fastest in terms of speed. The results also show that preprocessing steps such as stemming and stoplisting can significantly improve the performance of the IR system.

²F-measure: It is a statistical metric that balances precision (relevance of information) and recall (completeness of information).

4 Appendix

$$\text{Maximum TF Normalisation: } ntf(t, d) = a + (1 - a) \frac{tf_{t,d}}{tf_{max}(d)} \quad (1)$$

d is a smoothing factor $0 < d < 1$, typically 0.4, and

$tf_{max}(d) = \max_{\tau \in d} tf_{\tau,d}$, where τ ranges over all terms in d

$$\text{Similarity Score: } sim(\vec{q}, \vec{d}) = cos(\vec{q}, \vec{d}) = \frac{\sum_{i=1}^n q_i d_i}{\sqrt{\sum_{i=1}^n q_i^2} \sqrt{\sum_{i=1}^n d_i^2}} \quad (2)$$

No stoplist, No stemming			
	Binary	TF	TFIDF
Rel_Retr	47	44	119
Precision	0.07	0.07	0.19
Recall	0.06	0.06	0.15
F-measure	0.07	0.06	0.17
Time (sec.)	0.43	0.72	1.37

No stemming, With stoplisting			
	Binary	TF	TFIDF
Rel_Retr	84	99	125
Precision	0.13	0.15	0.20
Recall	0.11	0.12	0.16
F-measure	0.12	0.14	0.17
Time (sec.)	0.13	0.17	0.36

With stemming, No stoplisting			
	Binary	TF	TFIDF
Rel_Retr	61	68	151
Precision	0.10	0.11	0.24
Recall	0.08	0.09	0.19
F-measure	0.08	0.09	0.21
Time (sec.)	0.43	0.77	1.37

With stoplist, With stemming			
	Binary	TF	TFIDF
Rel_Retr	103	121	162
Precision	0.16	0.19	0.25
Recall	0.13	0.15	0.20
F-measure	0.14	0.17	0.23
Time (sec.)	0.29	0.25	0.53

Table 2: Results of three weighting systems under four different configurations

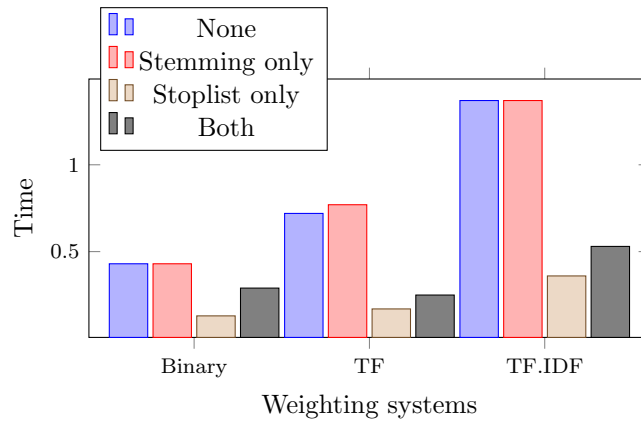


Figure 2: Time Retrieval of IR system under a range of configurations.